

BI / read / 20

query	BI / read / 20				
title	Recruitment				
pattern	Compute weighted shortest path between all Persons who work in the Company to Person person2 on knows.weight company: Company name = \$company workAt person1: Person # person2 person2: Person id = \$person2Id compute weighted shortest path on knows.weight knows.weight: min(abs(studyAtA.classYear - studyAtB.classYear)) + 1 personA: Person knows personB: Person studyAtA: studyAt University studyAtB: studyAt Example for finding a path between person1 and person2 p1 knows pX knows pY knows ... knows pW knows p2 studyAt University studyAt University studyAt University studyAt University studyAt University				
description	Consider knows edges where the endpoint Persons attended the same University and set the weight of the edge to the absolute difference between the year of enrolment plus 1. If the Persons attended multiple universities, we select the smallest (min) value. Formally: $w = \min_{\text{studyAt}_A, \text{studyAt}_B} \text{studyAt}_A.\text{classYear} - \text{studyAt}_B.\text{classYear} + 1$ Given a \$company and a Person person2 with ID \$person2Id (who is not working and has not worked at \$company), find a different Person (person1) who works or at some point worked in \$company and is reachable from person2 through people who have studied together through the shortest weighted path. If there are multiple Person person1 nodes with the same shortest path length, return all of them.				
params	1 \$company Long String	Companies with a similar number of employees (former or current) are selected			
	2 \$person2Id ID	(a) There is guaranteed to be no path between any person1 working at company and person2 (b) There is guaranteed to be a 2-hop path between at least one person1 working at company and person			
result	1 person1.id ID	R			
	2 totalWeight 32-bit Integer	C			
sort	1 totalWeight ↑				
	2 person1.id ↑				
limit	20				
CPs	3.3, 7.6, 7.7, 7.8, 8.4, 8.6				
relevance	To find the weighted shortest paths efficiently, the system can use e.g. a bidirectional Dijkstra algorithm. As the edge weights do not depend on any parameter, systems can pre-compute them (if they do not interleave reads and writes).				